

TET technical paper

Testnet v0.2, as the code stands · 2026-10-09 · written against commit

[04667cd98ffc8e4f87doac9db3669cd888ba571b](#) · the public seeds run 8239d10 · this is a testnet; data may be reset

TET is a public test network for checking who signed something, and when. Every transaction and signed record carries two signatures, Ed25519 and ML-DSA-44; members can post anonymously with a RISC Zero membership proof; messages and files are end-to-end encrypted with X25519 and CRYSTALS-Kyber-768 (Round 3). This paper describes what the code does today, including what it does not protect. Section 8 is the only part about plans.

1. Architecture

A node is one Rust binary, `TET-Core`. On start it reads its configuration from the environment, opens its libp2p and ML-DSA keystores, opens the ledger database, creates the genesis state if the chain is empty, restores its mempool, and starts three libp2p swarms (the block plane, the inference plane and the ledger plane), a swarm health watchdog, the block producer if `TET_AUTO_MINE` is set, and the REST server (port 5010 by default).

Storage is one sled database per node (`TET_DB_DIR`). The ledger keeps balances, blocks by height and by id, the canonical chain, undo records for reorgs, a transaction index and pending transactions in separate trees. Messages, files, the signature registry, sites and the operator's hide list live in the same database but outside the ledger: they are not part of the state root and are not on the chain.

A block (`BlockRecordV1`) records its height, block id, parent id, producer id, the transaction hashes and transactions, the state root and the reward. Its id is SHA-256 over `TET_BLOCK_ID_V2 |`, the height (u64, little-endian), `|parent=`, the parent id, `|state=`, the state root, `|txs=`, the comma-joined transaction hashes and `|producer=`, the producer id. There is no Merkle transaction root, and the id commits to no timestamp and no signature. The state root is SHA-256 over the sorted balance rows: it covers balances only. The default block time is 10 seconds; the seeds run 12.

A transaction is a `SignedTxEnvelopeV1`: the transaction (`TxV1`, one of `transfer`, `signer_link`, `founding_member_enroll`, `genesis_bridge`, `initial_airdrop`, `file_fee`, `worker_register`, `enterprise_inference`, `verify_zk_proof`), a hybrid

signature and an attestation. Its hash is SHA-256 of its chain-bound signing message (Section 2). Since commit 01d7192, a transaction must be signed by the wallet it acts for; this is checked when it is admitted, when a block is previewed and when a block is applied. The mempool holds up to 10,000 transactions or 64 MiB and evicts the lowest fee.

The block plane uses TCP with Noise and Yamux, gossipsub in strict mode with signed messages, Kademia, identify and mDNS. Gossip topics: `/tet/v1/blocks`, `/tet/v1/txs`, `/tet/v1/tmail`, `/tet/v1/files/announce`, `/tet/v1/ai-workload`. Request-response protocols carry chain hellos, catch-up ranges (up to 100 blocks or 8 MiB), single blocks by id, anonymous-membership data and file fetches. If the swarm's event loop stalls for 180 seconds, the watchdog exits the process with status 70 so its supervisor restarts it.

In public mode (`TET_PUBLIC_API`), the REST server answers only an allow-list of routes: chain status and explorer reads, the signature registry, sites, Tmail, Files and the anonymous-membership routes. Administrative routes answer only on loopback. Each client address gets token buckets: 10 reads a second (burst 40) and 20 writes a minute (burst 10), plus a daily upload budget.

Sources: [tet-core/src/main.rs](#), [tet-core/src/ledger.rs](#), [tet-core/src/consensus.rs](#), [tet-core/src/protocol.rs](#), [tet-core/src/p2p.rs](#), [tet-core/src/sync.rs](#), [tet-core/src/swarm](#) [health.rs](#), [tet-core/src/rest/public_api.rs](#), [tet-core/src/rest/state.rs](#)

2. Cryptography

Every signed object carries a hybrid signature: an Ed25519 signature and an ML-DSA signature over the same bytes, and both must verify. Ed25519 is `ed25519-dalek 2.2`. ML-DSA (FIPS 204) is `dilithium-rs 0.2.0`, a single-maintainer crate with no published third-party audit; the node vendors NIST's ACVP ML-DSA-44 key-generation and signature-verification vectors as tests. ML-DSA signing is deterministic: its randomness is SHA-256 of `tet:mldsa44-signing-rnd:v1` and the message.

Domain separation. Most objects sign a `|`-joined string that starts with a tag naming the object and binds the chain: for a transaction, `tet tx`

`v1|chain_id=...|genesis_hash=...|mldsa=...|tx=...`; for a Tmail envelope, `tet tmail envelope`

`v1|chain_id=...|genesis_hash=...|msg_id=...|...|payload_sha256=...|mldsa_pk=...`; for a site edit, `tet site edit v1|...`. A signature for one chain, or for one kind of object, does not verify as another. These strings are not length-prefixed. Signature records and

agent payloads use PAE (DSSE-style pre-authentication encoding, length-prefixed): `PAE("tet agent payload v1", [chain_id, genesis_hash, payloadType, payload])`, with payload types `application/vnd.tet.sha256` (a file's hash), `tet sig publish v1` (the signer's consent to publish a record) and `tet agent manifest v1`.

ML-DSA level. New keys are ML-DSA-44. Signature records, agent payloads and the demo's fee sponsor accept only ML-DSA-44 (1312-byte keys, 2420-byte signatures). Transactions, Tmail, sites and files still accept a key of level 44, 65 or 87, chosen by its length.

Wallet ID today. A wallet comes from 12 (or 24) BIP39 words. The Ed25519 secret key is the initial 32 bytes of the BIP39 seed (empty passphrase, no derivation path), and the wallet ID is the hex of the Ed25519 public key. The ML-DSA key comes from the same seed: HKDF-SHA256 with info `tet:pqc:mldsa44-seed:v1` gives a 32-byte seed for deterministic key generation. The wallet ID commits only to the Ed25519 key: the ML-DSA key is not bound to it. Binding both keys (`wallet_id_v2`) is a Phase 1 change (Section 8); until then the post-quantum half of a signature does not tie it to the wallet (SECURITY.md).

Hashes are SHA-256 throughout: transaction hashes, block ids, record hashes, site edit chains, commitments and nullifiers. A proof code `TET-XXXX-XXXX` is the leading 40 bits of a signature record's SHA-256, in Crockford base32. It is a lookup key, not a proof: the record's two signatures are what is checked.

In the browser, Ed25519 is `@noble/ed25519` and `@noble/curves`; ML-DSA is `tet-pqc-wasm`, the same `dilithium-rs` compiled to WebAssembly. A page checks records and envelopes itself: it rebuilds the signed bytes, pins ML-DSA-44 sizes and verifies both signatures, against the chain id and genesis hash it reads from the node (or that the reader types in). Tests check that the node, the WebAssembly module and the browser derive the same keys from the same words.

Sources: [tet-core/src/wallet.rs](#), [tet-core/src/quantum_shield.rs](#), [tet-core/src/fips204_vectors.rs](#), [tet-core/src/agent.rs](#), [tet-core/src/sigs.rs](#), [tet-core/src/rest/helpers.rs](#), [tet-pqc-wasm/src/lib.rs](#), [tet-network/ui/app/lib/proof_code.ts](#), [tet-network/ui/app/lib/ed25519_tet.ts](#), [SECURITY.md](#)

3. Anonymous membership

Joining. A member registers a commitment, SHA-256 of `tet-anon-v1` and a 32-byte member secret, in a record signed by their wallet. The secret is derived from the wallet's

words (HKDF-SHA256, info `tet-anon-member-v1`) and never leaves the member's device. Registration is free and the registry is node-local, off the chain. Every 60 seconds the node starts a new epoch; a registration counts from the next epoch. The members form a Merkle tree of depth 20 (up to 1,048,576 leaves), ordered by wallet id, with parent `SHA-256("tet-node-v1" || left || right)`.

The proof. A RISC Zero zkVM guest (risco-zkvm 3.0.5) proves, using SHA-256 only, that the prover knows a secret whose commitment is a leaf under a given root. Its public output (journal, kind `TAM1`) is the root, a nullifier, a one-time Ed25519 key that signs the post, the receiver (a board or a person) and the UTC day. The nullifier is `SHA-256("tet-null-v1" || secret || receiver || day)`, with the day as a little-endian u64 of `sent_at_ms / 86,400,000`. The prover runs on the member's own machine (127.0.0.1:9945), because it receives the secret.

Checks on the node. The envelope's one-time key must be the journal's, and its receiver and day must match. The receipt (about 250 KiB, fetched separately from gossip) must hash to the announced value, come from TET's pinned membership program (`ANON_GUEST_IMAGE_IDS`, or an id the operator adds), and verify. The root must be one the node built for an epoch within the last hour, for a day within one of today. The nullifier is then claimed: one message per nullifier. Since commit f5f968a the node does all of this before it stores or relays a post; a post that fails is refused, and one that arrived another way is deleted.

What follows. A member can post anonymously once per board per UTC day (up to three around midnight UTC, because a day either side is accepted). A post's daily ID is the opening four hex digits of its nullifier, shown only once the proof is verified: the same for one member on one board for one day, different on the next. Members-only polls use a member list chosen by the poll's creator (3 to 1,000 wallets); the node builds that list's root from its own registry, so a poll can't be filled with invented members.

Limits. A member is anonymous among the registrations their node has seen; on today's testnet that set is small, and so is the anonymity. Registration is free, so the set has no sybil resistance yet. The node sees the poster's IP address and timing.

Sources: [methods/guest/src/main.rs](#), [nexus-protocol/src/lib.rs](#), [prover/host/src/main.rs](#), [tet-core/src/tmail/anon.rs](#), [tet-core/src/tmail/store.rs](#), [tet-core/src/tmail/poll.rs](#), [tet-core/src/tmail/envelope.rs](#), [tet-network/ui/app/lib/anon_tree.mjs](#), [tet-network/ui/app/lib/board.mjs](#)

4. Tmail and Files

Key exchange is X25519 plus CRYSTALS-Kyber-768 Round 3, which is not FIPS 203 ML-KEM: the two are byte-incompatible. The node uses `pqcrypto-kyber` 0.8.1 and the browser `crystals-kyber-js` 1.1.2 (with `x25519-dalek` 2 and `@noble/curves`). Fields named `mlkem_*` are legacy names for these Kyber Round 3 keys. Moving both planes to FIPS 203 ML-KEM-768 is a Phase 1 item.

Encryption. For each message the sender makes a fresh X25519 key and a fresh Kyber encapsulation to the receiver's keys; HKDF-SHA256 over both shared secrets, with info `tet-e2ee-hybrid-v1`, gives a 32-byte key for ChaCha20-Poly1305 with a 12-byte nonce. Messaging keys come from the wallet's words (HKDF labels `tet-tmail-x25519-v1` and `tet-tmail-mlkem-v1`) and are published in a record signed by the wallet.

The envelope is signed by the sender's wallet, or, for an anonymous post, by the one-time key in its proof. The signature covers the message id, flags, sender, receiver, release time, fee and the SHA-256 of the ciphertext. Burn after reading is a signed request, by the sender or the receiver, that cooperating nodes delete the message: best effort, not cryptographic erasure. A time lock is a signed release time before which cooperating nodes withhold the ciphertext; anyone holding the receiver's keys and the ciphertext could read it earlier.

Retention. Messages are kept 7 days by default and 30 at most; a named sender's newest 5 per conversation, a receiver's newest 100 anonymous messages (all of a poll's ballots), and 50,000 messages in all. Files use the same key exchange (HKDF info `tet-file-v1`), with the name, type and body encrypted separately; the code's defaults are 5 MB a file and 30 days, and the demo node is configured for 100 MB and 7 days.

What a node sees: the receiver, the sender's wallet (or `anonymous` and the one-time key), the times, the flags, the sizes and, for an anonymous post, its journal. It never sees the plaintext, and there is no decryption route.

Sources: [tet-core/src/e2ee.rs](#), [tet-core/src/tmail/envelope.rs](#), [tet-core/src/tmail/keys.rs](#), [tet-core/src/tmail/burn.rs](#), [tet-core/src/tmail/timelock.rs](#), [tet-core/src/tmail/store.rs](#), [tet-core/src/files/mod.rs](#), [tet-core/src/files/storage.rs](#), [tet-network/ui/app/lib/tmail_e2ee.ts](#), [tet-network/ui/app/lib/tmail_keys.ts](#), [tet-network/ui/app/lib/files_e2ee.ts](#), [tet-network/ui/package.json](#)

5. Consensus today

TET today has one block producer. The Helsinki seed is the only validator, by explicit configuration (`TET_VALIDATOR_IDS`); the other seed and any other node follow it. The code has leader election for several validators, but with one validator the leader is always the same node. One producer can censor, reorder or delay transactions, and decides when blocks happen.

A follower applies a block only if the height continues its chain, the producer is in its validator set and is the expected leader, the parent matches, no transaction repeats, the block id recomputes, the reward is right, and the state root matches both before and after applying. Everything in that list can be recomputed by a follower, so a producer can't make a follower accept an invalid balance change.

Blocks carry no producer signature yet, and the producer id in a block is an unsigned string. What authenticates a block's source today is the producer's libp2p identity: followers pin the producer's PeerId (`TET_PRODUCER_PEERS`). For gossip, gossipsub in strict mode authenticates the message author; for sync (catch-up ranges and blocks fetched by id), a follower takes blocks only from trusted peers, its pinned producer or relays its operator lists. The sole producer takes blocks from no peer unless its operator names one. Producer signatures arrive in Phase 1.

Forks resolve to strictly greater cumulative weight; otherwise the local chain is kept. The block time is a timer on the producer's clock, and vesting locks compare against the local clock when a block is applied: wall-clock time is a consensus input (`SECURITY.md`).

Sources: [tet-core/src/consensus.rs](#), [tet-core/src/p2p.rs](#), [tet-core/src/sync.rs](#), [tet-core/src/ledger.rs](#), [SECURITY.md](#)

6. Threat model

What TET protects against today:

- Changing signed content: a node or relay that alters a transaction, message, record or site edit breaks its signatures, which every node and every reader's page check.
- Acting for someone else's wallet: a transaction must be signed by the wallet it acts for. Today that means its Ed25519 key, because the wallet ID is the Ed25519 public key and the ML-DSA key isn't bound to it; once `wallet_id_v2` binds both keys (Phase 1), forging a transaction will require breaking both Ed25519 and ML-DSA-44.

- Replaying a signature on another chain or as another kind of object: the chain id, genesis hash and an object tag are signed.
- A member posting anonymously twice on one board in one day, and anonymous posts whose proofs don't verify being stored or passed on.
- Reading messages and files: they are end-to-end encrypted; nodes and relays hold ciphertext only.
- Blocks from peers other than the pinned producer, by gossip or by sync.

What it does not protect against:

- A quantum adversary, completely: quantum resistance is incomplete until `wallet_id_v2` (Phase 1). Today an attacker who could break Ed25519 could sign for a wallet with an ML-DSA key of their own. Message key exchange uses Kyber Round 3, not the final ML-KEM.
- The node seeing IP addresses, timing and who messages whom (for named messages). Use Tor or your own node for IP privacy.
- The single producer censoring, reordering, delaying or halting the chain, or its clock deciding time-dependent rules.
- Who holds a key, who wrote something, whether it is true, or whether AI was used. A mark proves when, and whose mark; not who the author is.
- Anonymity beyond the small set of registrations a node has seen; a determined observer with network access can correlate timing.
- Data loss: this is a testnet, data may be reset, and messages and files expire.
- Bugs: nothing has been audited, and the post-quantum libraries are young.

Sources: [SECURITY.md](#), [tet-core/src/protocol.rs](#), [tet-core/src/rest/helpers.rs](#), [tet-core/src/tmail/anon.rs](#), [tet-core/src/p2p.rs](#)

7. Known weaknesses and security history

SECURITY.md is the public list, kept with the code. Known limitations today include: no audit; a young signature stack; two seeds and one producer; a producer that has stalled before (for 33 hours, with a postmortem); identity binding that rests on Ed25519 alone; blocks authenticated by PeerId rather than a producer signature; weak anonymity by construction; some balance writes outside consensus; wall-clock time as a consensus input; invalid zero-knowledge receipts refused but not punished; and a development chain id on the testnet.

Fixed issues, described by class (the details are kept in a private repository):

- The desktop's anonymous mode could send a named message (fixed before PR #17).
- A transaction did not have to be signed by the wallet it acts for (fixed and deployed 2026-10-08; the chain was checked and had no such transaction).
- An anonymous post whose proof did not verify could still be kept and passed on (fixed and deployed 2026-10-09).
- Blocks received during chain sync were not held to the producer pin (fixed and deployed 2026-10-09).

To report a vulnerability, follow SECURITY.md: privately, not in a public issue.

Sources: [SECURITY.md](#), [docs/postmortems/2026-10-04-producer-wedge-33h.md](#)

8. Phase 1

Phase 1 is a new genesis. Its target is Q1 2027: a target published as a quarter, not a promise; if it moves, it moves in public. None of the items below runs on the testnet today.

Change	What it does	State
V3 block header	The producer's timestamp (<code>ts_ms</code> , strictly after the parent's; a block more than 60 s ahead is held) is covered by a length-prefixed (PAE) block id.	Built on the Phase 1 branch
Producer signatures	A dedicated hybrid Ed25519 + ML-DSA-44 producer key, never a wallet key, signs each block id; validators are named in the genesis; checked for gossip, catch-up and backfill.	Built on the Phase 1 branch
Per-plane keys	Separate libp2p identities for the block, inference and ledger planes, derived with HKDF.	Built on the Phase 1 branch
<code>wallet_id_v2</code>	The wallet ID commits to both keys: <code>hex(SHA-256(PAE("tet wallet id v2", [ed25519_pk, mldsa44_pk])))</code> , with ML-DSA pinned at level 44. Changes every wallet id.	Designed, not built
Move to FIPS 203 ML-KEM-768	Move both encryption planes from Kyber Round 3 to the final standard. Changes every messaging key.	Open

Not in Phase 1: more block producers. Changing the validator set is planned after the Phase 1 genesis; until then TET has one producer, as Section 5 says.

Sources: [docs/PHASE 1 GENESIS SPEC.md](#), [docs/QUEUE.md](#), [SECURITY.md](#)

This paper's proof code

TET-W78E-P3YG



The code marks the text of this paper: the file `paper/tet-technical-paper.html`, SHA-256

`c2c6428ad4430e248d6024c98a762a10f15785bdac889c56537a8b89c6410a6a`,
marked by TET's publisher ID `0251cee93e087e90715c164e975ee929ee9a94cc2abd95c97b1e62fa547339b2`.

Check it: enter the code at try.stevenexus.org, or scan the QR. Or drop that HTML file into the search box: an exact copy matches; a copy changed by one byte doesn't.

What it proves: this ID marked exactly that text. What it doesn't prove: who wrote it, or that it is correct. This PDF file has a code of its own, shown next to its download link (a file can't contain its own code).